VIBE CODING

生成式软件开发（5）

– 规划先行：Plan模式

Roadmap

Level 1:Chat-Driven(会话驱动)

- **适合**:简单探索、一次性小脚本。
- **核心**: 会话->执行
- **缺点**:上下文容易漂移,做不了复杂任务。
- **特点**:没有文档

Level 2:Plan 模式(任务驱动)

- **适合**:复杂的开发任务。
- **核心**:单次任务计划书 -> Review 对齐 -> 执行。
- **缺点**: 无法知道系统的全貌
- **特点**: 每一个任务一个文档,执行完并确认后不再维护

Level 3:Spec-Driven Development(SDD)(规范驱动)

- **适合**:长期维护、具备复杂业务深度的系统。
- **核心**:统一的文档(API、架构、数据模型) -> Review -> 执行。
- **缺点**: 需要很强的规划技巧。
- **特点**: 文档先行,持续维护 ,代码可以认为从文档生成。

已完成 → 本周.... → 下半学期

大纲

01

会话驱动模式的问题

理解会话驱动开发的优缺点及其适用场景，分析为什么不适合复杂系统

02

什么是 Plan 模式

Plan 模式的定义、原则与核心优势，对比功能迭代与架构调整场景

03

Plan 模式的最佳实践

Plan 文档结构、防腐化实践、版本管理与变更应对策略

04

案例演示

展示完整的开发流程与工作流程示例

核心目标：让 AI 从"聊天对象"变成"基于规范执行的助手"

01

会话驱动模式的问题

理解会话驱动开发的优缺点及其适用场景

会话驱动开发的特征与局限

什么是会话驱动开发

无论是在浏览器直接与大模型会话，还是在 Agentic IDE 中通过会话完成开发，本质上都是**基于会话的开发**。

特征：通过会话一步步执行，每次都看到结果，再决定下一步的内容。

优点

- **即时反馈**
  快速看到初步结果，便于调整方向
- **灵活应变**
  根据中间结果动态调整需求
- **降低门槛**
  无需预先规划完整方案
- **渐进理解**
  在会话中逐步明确需求细节

缺点

- **上下文漂移**
  多轮会话后容易偏离最初目标
- **难以复现**
  缺乏文档记录，无法复现开发过程
- **质量不稳定**
  结果质量取决于会话技巧和运气

适用场景

场景	推荐度	说明
探索性原型	V 推荐	快速验证想法可行性
简单脚本	V 推荐	一次性任务，无需维护
学习新技术	V 推荐	通过会话理解概念
复杂系统开发	✗ 不推荐	需要结构化规划
团队协作项目	✗ 不推荐	需要可文档化的规范
长期维护项目	✗ 不推荐	需要可追溯的设计决策

关键洞察：会话模式适合快速验证和简单任务，但对于需要长期维护、多人协作的复杂系统，需要更结构化的方法。

为什么会话驱动的模式不适合复杂系统

1 上下文与注意力机制的物理瓶颈

即使模型拥有几百万 Token 的上下文窗口，AI 仍存在明显的 **"迷失"（Lost in the Middle）** 现象。

随着会话轮次增加，注意力发散，极易遗忘早期定义的全局约束和核心业务逻辑。

2 缺乏全局重构的能力

人类工程师

评估现有架构，进行 **防御性重构**（抽离接口、重组依赖）后再添加新功能

会话式 AI

倾向于 **"打补丁"（Patching）**，代码陷入无法演进和维护的状态

核心问题：会话模式缺乏全局视野和系统性思考能力，难以应对复杂系统的演进需求

02

什么是Plan 模式

Plan 模式的定义、原则与核心优势

什么是Plan模式

Plan 模式是一种结构化的 AI 协作开发方法

- 每个项目都可以分解为很多子任务
- **对于一个复杂的子任务，先与 AI 一起制定详细的计划文档（Plan），仔细Review, 再基于该计划生成代码，**而不是直接通过简单的会话让 AI 写代码。

1 先规划，后编码

- 不急于让 AI 开始写代码
- 先花时间明确需求、架构和约束
- 将思考结果沉淀为结构化的 Plan 文档

2 文档即契约

- Plan 是人与 AI 之间的"共识记录"
- 所有需求、规则，都明确写在文档中
- AI 基于文档生成代码，而非零散的记忆

3 变更驱动

- 需求、架构变更时先构建Plan文档
- 经过Review，从Plan 构建代码
- 可以保留迭代的正式记录

一个简单的示例



/ plan 优先规划任务的执行方向，用户确...

/ spec 根据需求细化完整的规范、任务、...

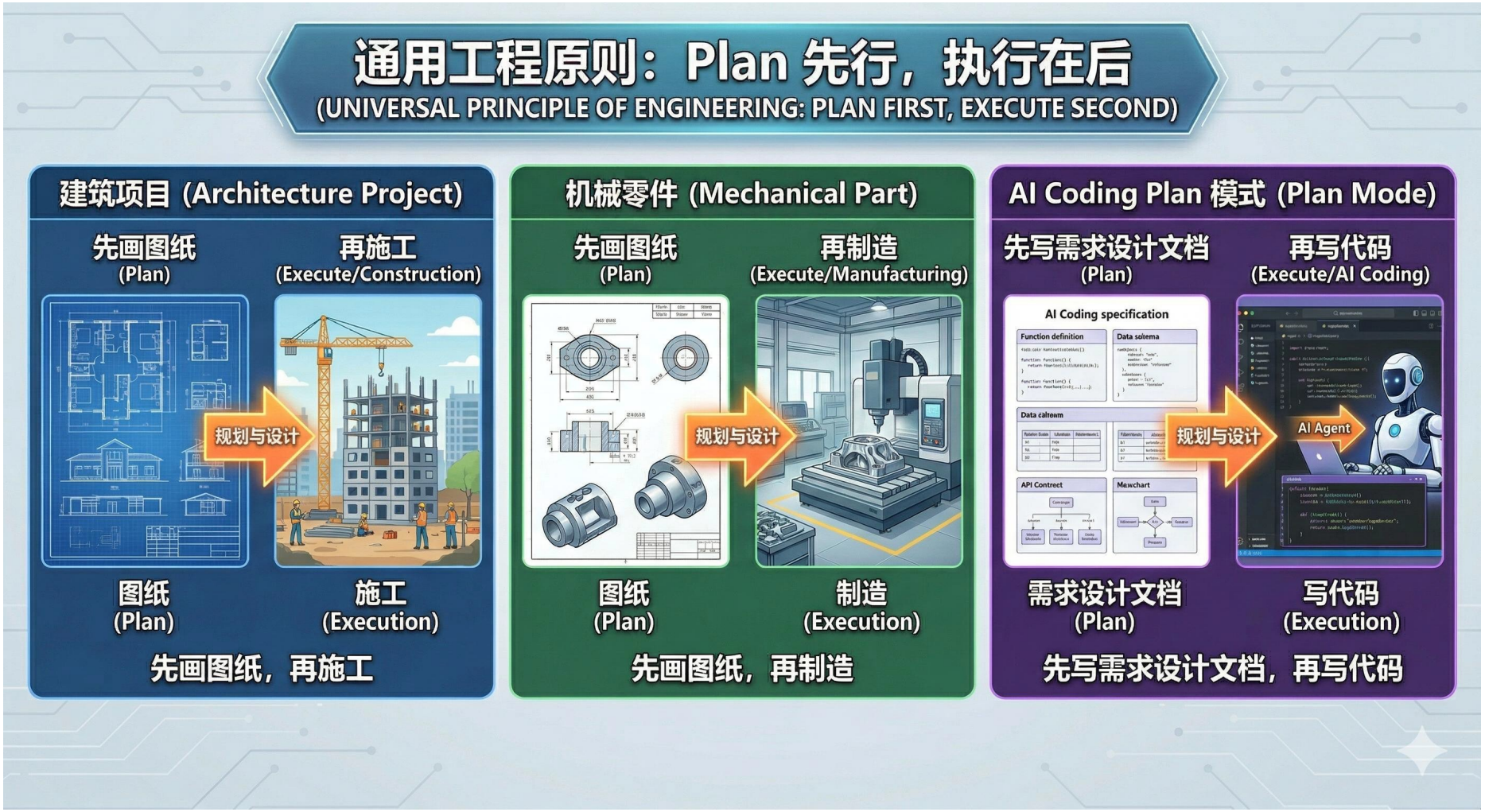
优先规划任务的执行方向，用户确认后再执行

/

@ #

MiniMax-M2.7

与其他工程领域对比


通用工程原则：Plan 先行，执行在后
(UNIVERSAL PRINCIPLE OF ENGINEERING: PLAN FIRST, EXECUTE SECOND)
建筑项目 (Architecture Project)
先画图纸
(Plan)
再施工
(Execute/Construction)
规划与设计
图纸
(Plan)
施工
(Execution)
先画图纸，再施工
机械零件 (Mechanical Part)
先画图纸
(Plan)
再制造
(Execute/Manufacturing)
规划与设计
图纸
(Plan)
制造
(Execution)
先画图纸，再制造
AI Coding Plan 模式 (Plan Mode)
先写需求设计文档
(Plan)
再写代码
(Execute/AI Coding)
AI Coding specification
Function definition
Data schema
Data structure
API Contract
Mewchart
规划与设计
AI Agent
需求设计文档
(Plan)
写代码
(Execution)
先写需求设计文档，再写代码

Plan模式：减少变更的频度

通过制定相对正式的计划，可以减少变更的频度

即便是 AI，也无法有效处理过于剧烈、缺乏规划的迭代。

提前发现矛盾

写代码前发现需求冲突或设计缺陷

明确边界条件

预先定义异常情况的处理方式

建立共识

与 AI 对"要做什么"达成一致后再动手

Plan 模式：模块化分解

模块化分解

Plan 的编写过程迫使我们在编码前思考：

模块拆分

如何将复杂系统拆分为相对独立的模块？

接口定义

各模块之间的接口如何定义？

数据流动

数据如何在模块间流动？

收益

- ✓ 每个模块职责清晰，便于单独开发和测试
- ✓ AI 可以分模块生成代码，降低单次任务的复杂度
- ✓ 问题定位更容易（模块边界明确）

Plan 模式与会话模式对比

维度	会话模式	Plan 模式
起点	"帮我写个功能..."	"这是我的 Plan，请基于此生成..."
信息载体	散落在多轮会话中	集中在结构化文档中
需求变更	口头描述，容易遗漏	文档更新，有据可查
代码质量	取决于会话技巧	取决于 Plan 的完整性
可维护性	难以复现和交接	文档即知识载体

核心洞察：Plan 模式不是禁止变更，而是尽可能让变更在可控的框架内发生

对比案例

场景一：功能迭代

会话模式

触发方式： 会话输入："增加按不同的角色限制对功能的访问"
过程： 基于角色的访问控制有不同实现方式和细节，AI自行选择。
结果： 生成代码细节后偏差，需要多轮修复

Plan 模式

触发方式： "/plan 增加按不同的角色限制对功能的访问"
过程： 多轮会话完善plan
结果： 一次生成符合预期的搜索功能，接口与数据模型保持一致

场景二：架构调整

会话模式

触发方式： 会话输入"发现数据存内存会溢出，需改成外部存储"
过程： 外部有不同实现方式和细节，AI自行选择。
结果： 生成代码细节后偏差，需要多轮修复

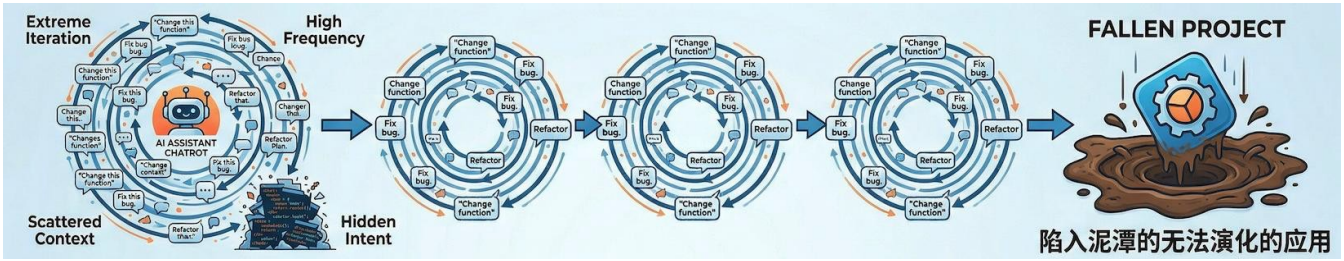
Plan 模式

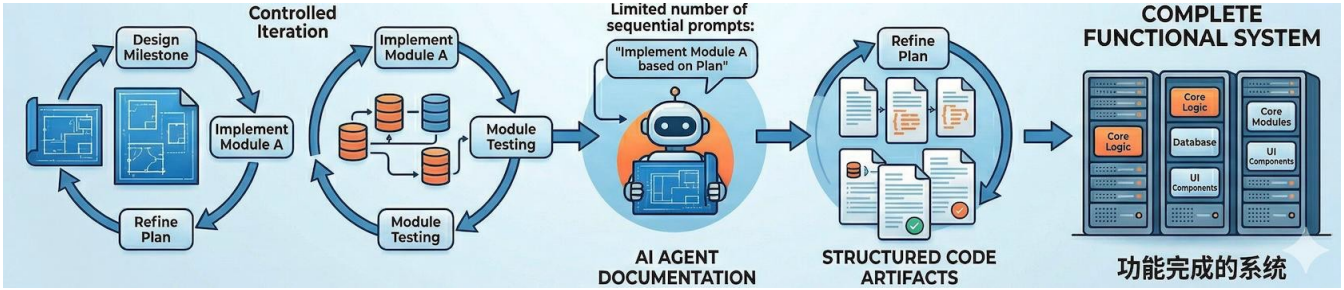
触发方式： /plan 发现数据存内存会溢出，需改成外部存储
过程： 多轮会话完善plan文档，含模块化设计的review
结果： 改动范围可控，其余模块无需调整，AI 生成的代码确定性高

由会话驱动到Plan驱动

会话模式

Plan模式





Plan 模式是一种更加高效的上下文工程实践

Plan 模式通过结构化的文档将项目背景、需求、约束等信息集中管理，使 AI 能够一次性获取完整、准确、无冲突的上下文。Plan 模式的核心产出是一组**结构化文档**（如 file_upload_plan.md, role_based_access_feat.md），它们：

- 是人与 AI 都能够理解的"契约"
- 是需求变更时的"变更基准"
- 是项目交接时的"说明文档"

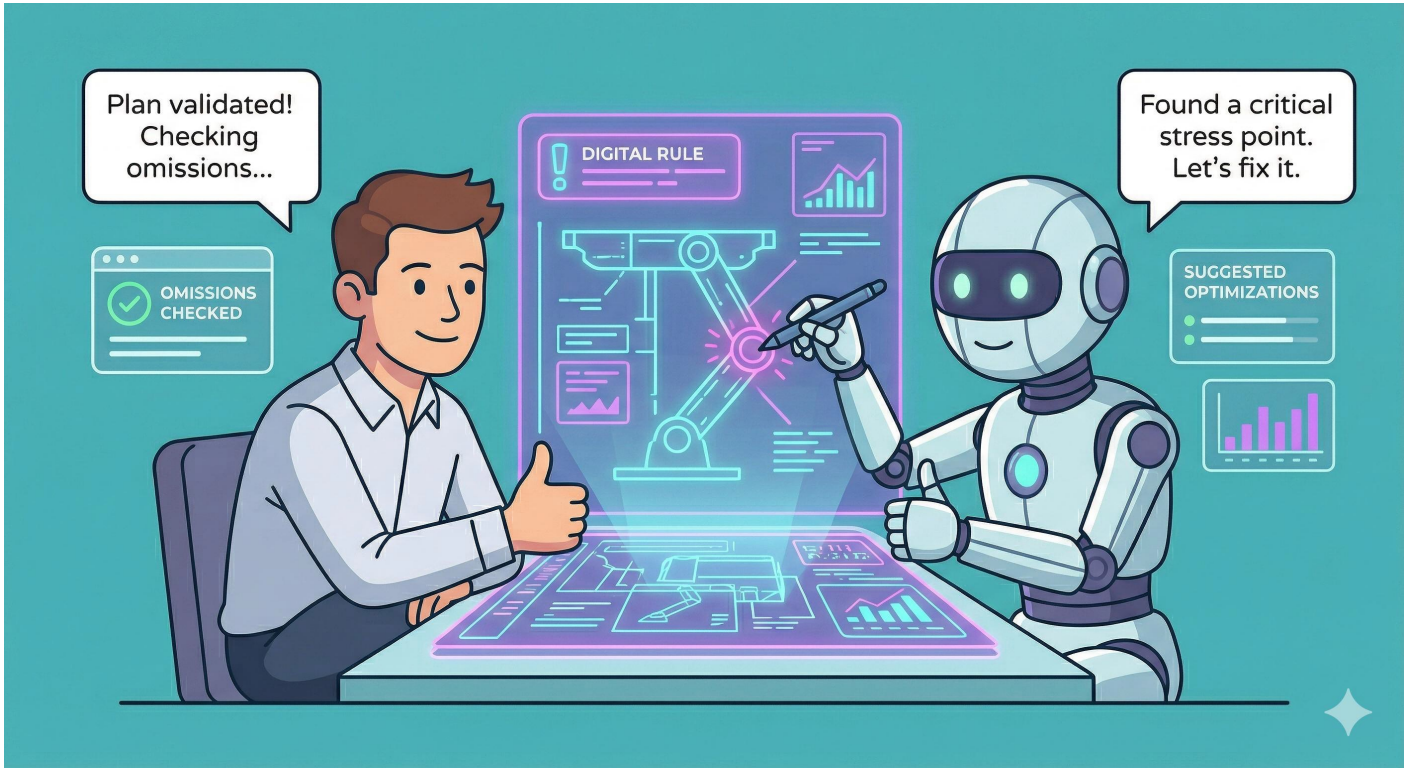


03

Plan 模式的最佳实践

AI 结对评审 (AI Pair Review)

核心思想： 写 Plan 的过程就是对齐思路的过程。在执行前，让 AI 扮演架构师/评审员，检查你的初始想法是否有遗漏（如边界条件、异常处理），并由 AI 补充细节，双方确认无误后再动手。



任务级别的文档驱动

核心思想： 在进入执行阶段前，Plan 可以与 AI 反复讨论修改；但一旦进入真正的编码执行阶段，Plan 就应被视为“冻结”，后续修改应直接针对代码。如果任务偏航太远，则终止任务，开启新的一轮 Plan。

Plan 模式的正确做法

1. **制作 Plan**

   与AI共同制定Plan的一个稳定版本

2. **提交 Plan 到暂存区(stash, `git add .`)**

   如果代码生成过程中发现plan有比较大的偏差，删除生成的代码，恢复(restore) plan，重新完善plan

3. **让 AI 基于 Plan 生成代码**

   这个过程中可以通过会话让AI修改一些小的细节或修复bug

4. **提交代码**

   `git commit -m "fea: 访问控制"`

分步执行与清单打钩

核心思想： 在 Plan 的末尾要求 AI 附带清晰的 Todo-list，执行时让 AI 逐条打钩确认，避免“一口气写完所有代码”导致失控。在必要的情况下，可以要求AI执行到某个步骤后先停下来，等待确认后继续。



基于Plan模式的开发流程

关键步骤

1 **构思**

(Draft)

用大白话写出最初的想法和需求

2 **结构化**

(Structure)

套用模板，转化为轻量级的 Plan

3 **修改提示与AI 评审**

(Refine)

让 AI 充当评审员，逐步完善 Plan

4 **执行**

(Execute)

将完整的 Plan 交给 AI 生成代码

5 **评审和迭代**

(Iterate)

根据结果开始下一轮，所有修改都写在 plan 文档中

工作流程示例

人 我想做个景点管理系统，需要...

AI 我建议从以下几个方面规划，您看是否完整？

人 还缺少权限控制的部分

AI 已补充，请查看更新后的 Plan

人 数据验证规则需要再细化...

AI 已细化，请再次确认

Roadmap

Level 1:Chat-Driven(会话驱动)

- **适合**:简单探索、一次性小脚本。
- **核心**: 会话->执行
- **缺点**:上下文容易漂移,做不了复杂任务。
- **特点**:没有文档

Level 2:Plan 模式(任务驱动)

- **适合**:复杂的开发任务。
- **核心**:单次任务计划书 -> Review 对齐 -> 执行。
- **缺点**: 无法知道系统的全貌
- **特点**: 每一个任务一个文档,执行完并确认后不再维护

Level 3:Spec-Driven Development(SDD)(规范驱动)

- **适合**:长期维护、具备复杂业务深度的系统。
- **核心**:统一的文档(API、架构、数据模型) -> Review -> 执行。
- **缺点**: 需要很强的规划技巧。
- **特点**: 文档先行,持续维护 ,代码可以认为从文档生成。

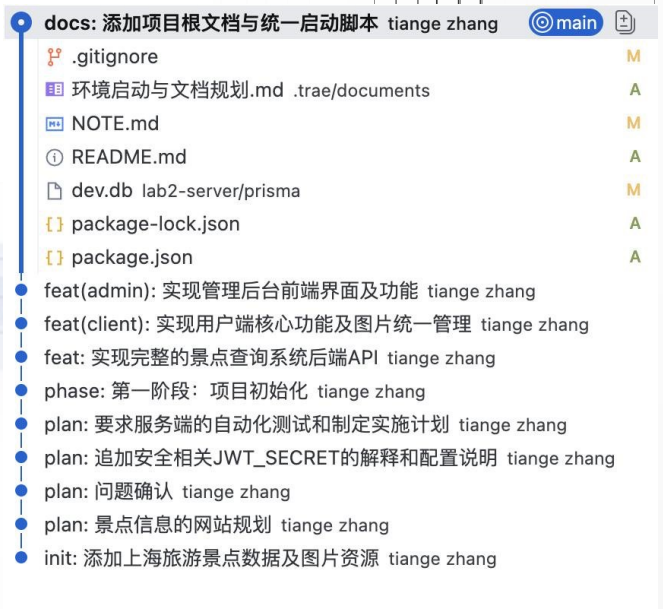
下半学期

C/S架构应用的开发

案例演示

展示完整的开发流程与工作流程示例





End

2.氛围编程的关键实践 (3)

- 结构化 AI 协作开发方法：Plan模式